Experiment 08: Automating Application Deployment using GitHub Actions / Jenkins

Experiment No.	08
Mapped CO	CO3
Aim	To implement an automated deployment pipeline using GitHub Actions or Jenkins.
Tools / Software	GitHub account / Jenkins; Docker Hub or a container registry; A deployment target such as Kubernetes or a server

Theory

Continuous Deployment (CD) is the practice of automatically releasing every validated software change to the target environment. After the CI stages such as build and test are successful, the deployment stage packages the application, publishes the build artifact or container image, and updates the running environment.

In a container-based deployment pipeline, the workflow commonly logs in to a container registry, builds a Docker image, tags it with a meaningful version such as a commit SHA, pushes it to Docker Hub or another registry, and then triggers deployment on Kubernetes or a remote server.

GitHub Actions implements deployment as code through YAML workflow files. Jenkins provides a similar automation mechanism using Pipeline jobs and Jenkinsfiles. Secrets such as Docker credentials, cluster credentials and server keys must be stored securely as GitHub repository secrets or Jenkins credentials and must never be committed to source control.

Procedure

1. Create or use an existing GitHub repository containing the application source code and Dockerfile.
2. Ensure the CI workflow already builds and tests the application successfully.
3. Create a Docker Hub account or use another container registry for storing the application image.
4. Store registry credentials securely as repository secrets, for example DOCKER_USERNAME and DOCKER_PASSWORD.
5. Create or update the workflow file in .github/workflows, for example deploy.yml or ci.yml.
6. Add a deployment job that runs only after the build-test job is successful and only on the main branch.
7. Add steps to check out the source code, log in to Docker Hub, build the Docker image and push it with a versioned tag.
8. Add a deployment step to update the target environment, for example by using kubectl set image for Kubernetes or a remote server restart command.
9. Commit and push the workflow file to the main branch.
10. Open the Actions tab or Jenkins build history and verify that the build, image push and deployment stages complete successfully.
11. Confirm that the latest deployed application is running by opening the deployment URL or executing a curl command.

Sample Code / Commands

GitHub Actions deployment job appended to ci.yml

```
deploy:
  needs: build-test
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: docker/login-action@v3
      with:
        username: ${{ secrets.DOCKER_USERNAME }}
        password: ${{ secrets.DOCKER_PASSWORD }}
```

```
    - uses: docker/build-push-action@v6
      with:
        push: true
        tags: ${{ secrets.DOCKER_USERNAME }}/myapp:${{ github.sha }}
```

Example deployment update commands

```
# Kubernetes target
kubectl set image deployment/myapp myapp=${{ secrets.DOCKER_USERNAME }}/myapp:${{ github.sha }}
kubectl rollout status deployment/myapp

# Server target example
ssh user@server 'docker pull <user>/myapp:<tag> && docker restart myapp'
```

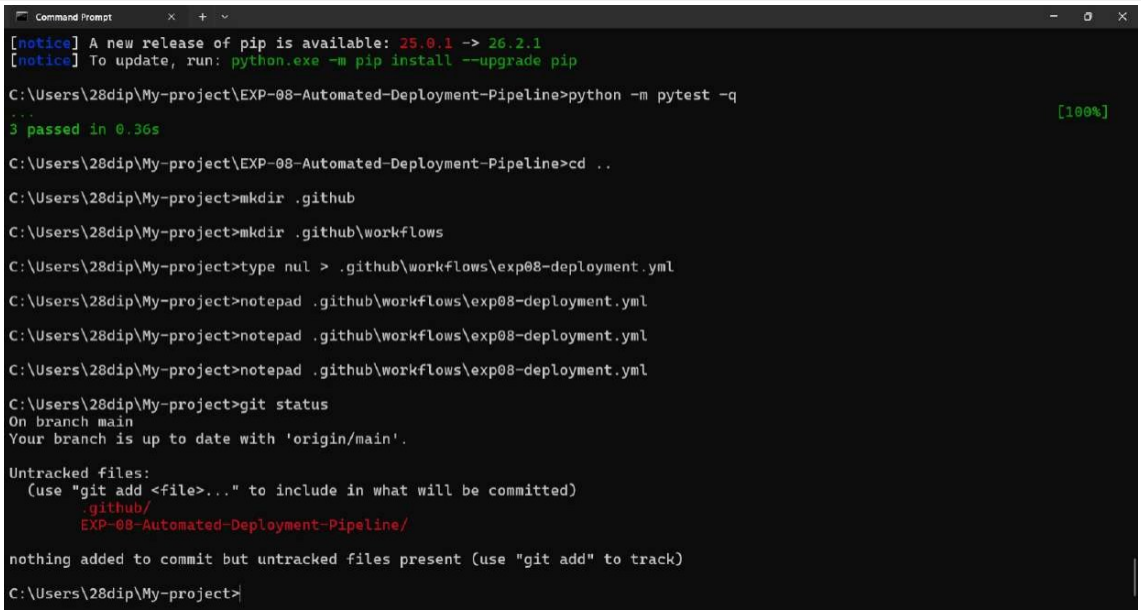
Observation / Experiment Output

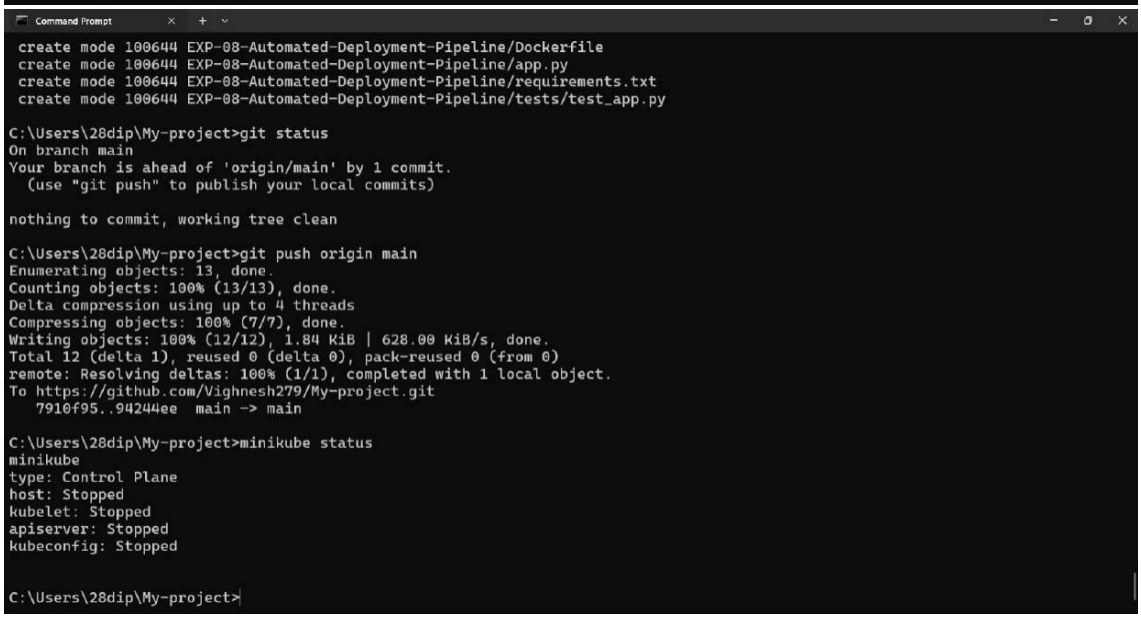
Instructions to students: Paste screenshots or copied outputs in the boxes below after completing the automated deployment pipeline using GitHub Actions or Jenkins. Keep screenshots clear and readable.

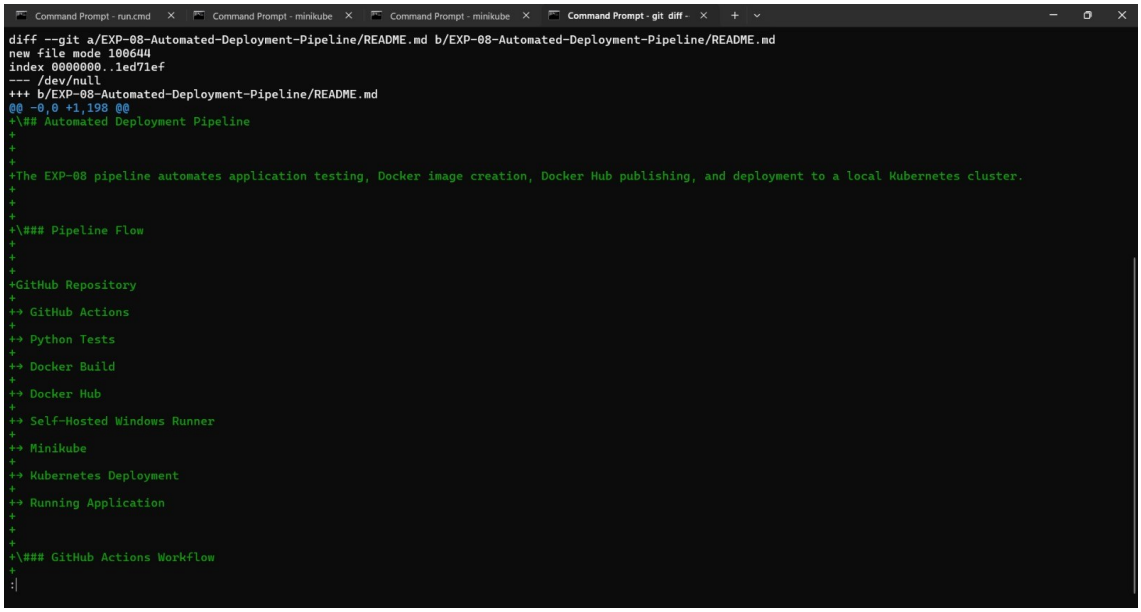
Name	Roll No.	Batch	Date
Vighnesh Pingale	24107108	B2	04-10-2026

1. Repository and Application Setup Output

Paste screenshot/output showing GitHub repository, application source code, Dockerfile and CI workflow file location.

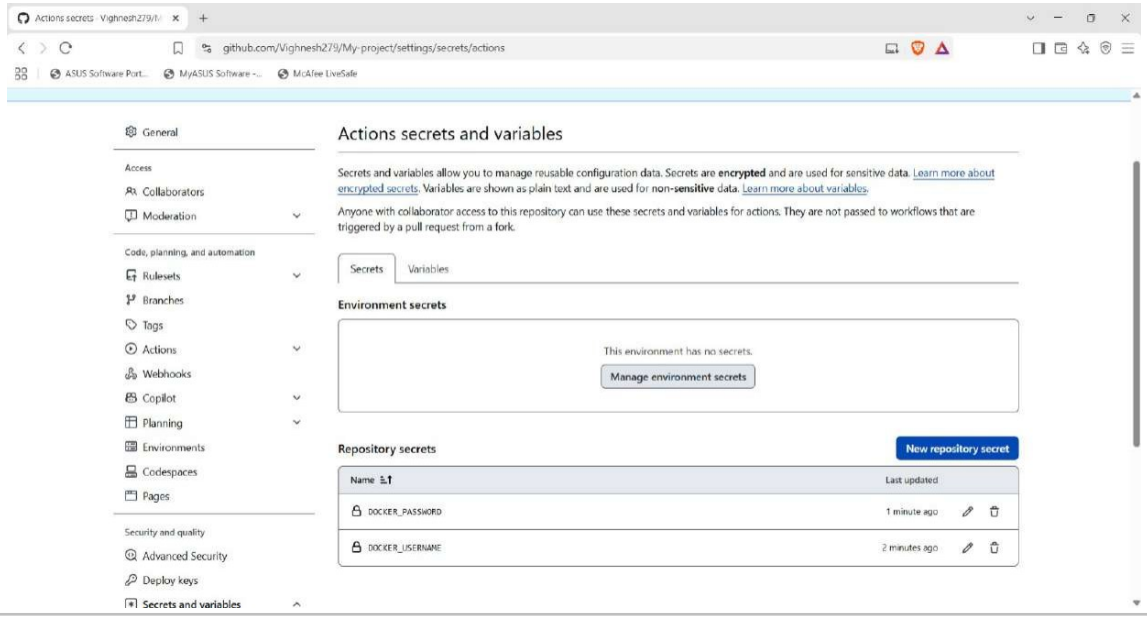






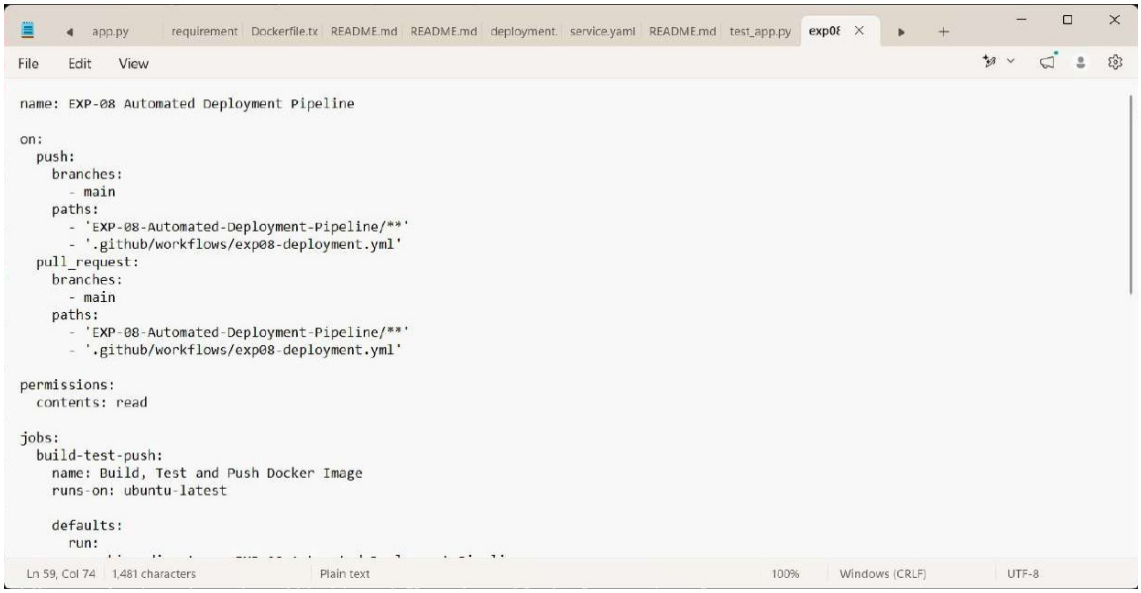
2. Repository Secrets / Jenkins Credentials Output

Paste screenshot/output showing configured repository secrets or Jenkins credentials. Do not reveal actual secret values.



3. Deployment Workflow YAML Output

Paste screenshot/output showing deploy job with build-test dependency, registry login, image build and image push steps.



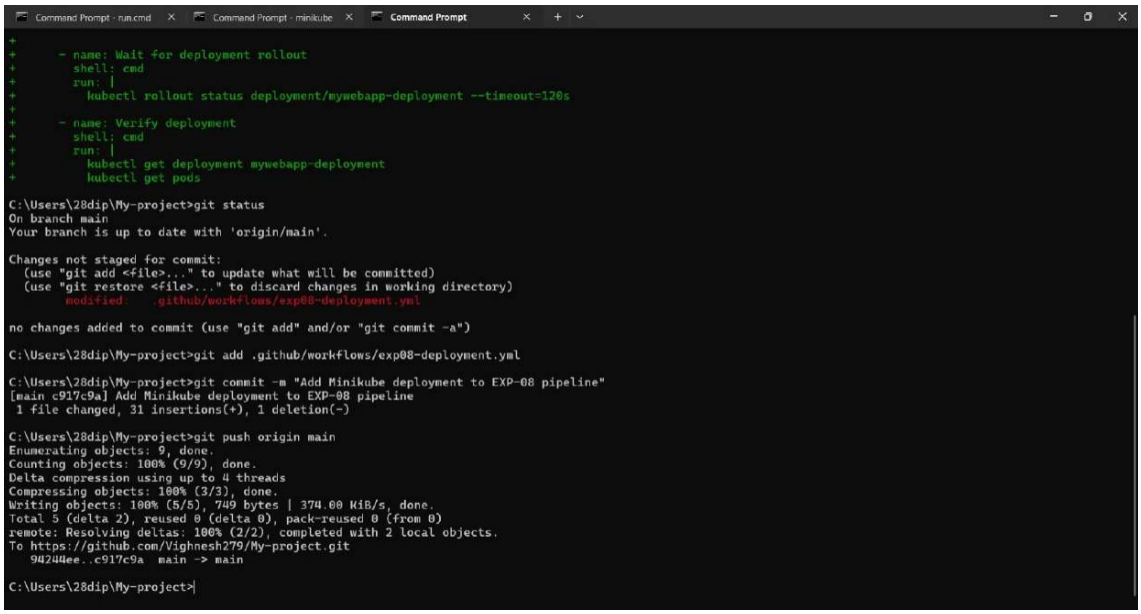
```
name: EXP-08 Automated Deployment Pipeline

on:
  push:
    branches:
      - main
    paths:
      - 'EXP-08-Automated-Deployment-Pipeline/**'
      - '.github/workflows/exp08-deployment.yml'
  pull_request:
    branches:
      - main
    paths:
      - 'EXP-08-Automated-Deployment-Pipeline/**'
      - '.github/workflows/exp08-deployment.yml'

permissions:
  contents: read

jobs:
  build-test-push:
    name: Build, Test and Push Docker Image
    runs-on: ubuntu-latest

    defaults:
      run:
```



```
+
+      - name: Wait for deployment rollout
+        shell: cmd
+        run: |
+          kubectl rollout status deployment/mywebapp-deployment --timeout=120s
+
+      - name: Verify deployment
+        shell: cmd
+        run: |
+          kubectl get deployment mywebapp-deployment
+          kubectl get pods

C:\Users\28dip\My-project>git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .github/workflows/exp08-deployment.yml

no changes added to commit (use "git add" and/or "git commit -a")

C:\Users\28dip\My-project>git add .github/workflows/exp08-deployment.yml

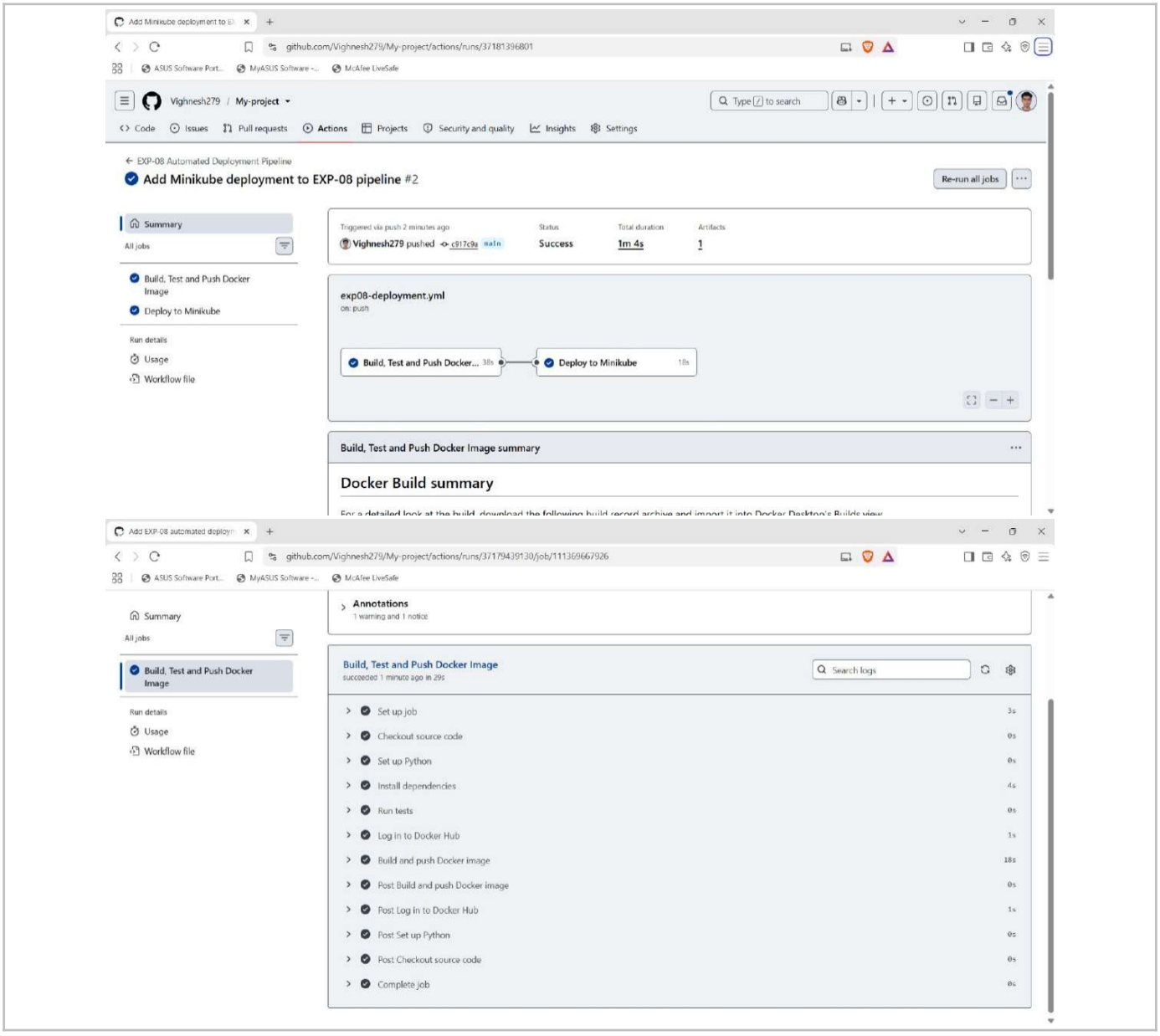
C:\Users\28dip\My-project>git commit -m "Add Minikube deployment to EXP-08 pipeline"
[main c917c9a] Add Minikube deployment to EXP-08 pipeline
 1 file changed, 31 insertions(+), 1 deletion(-)

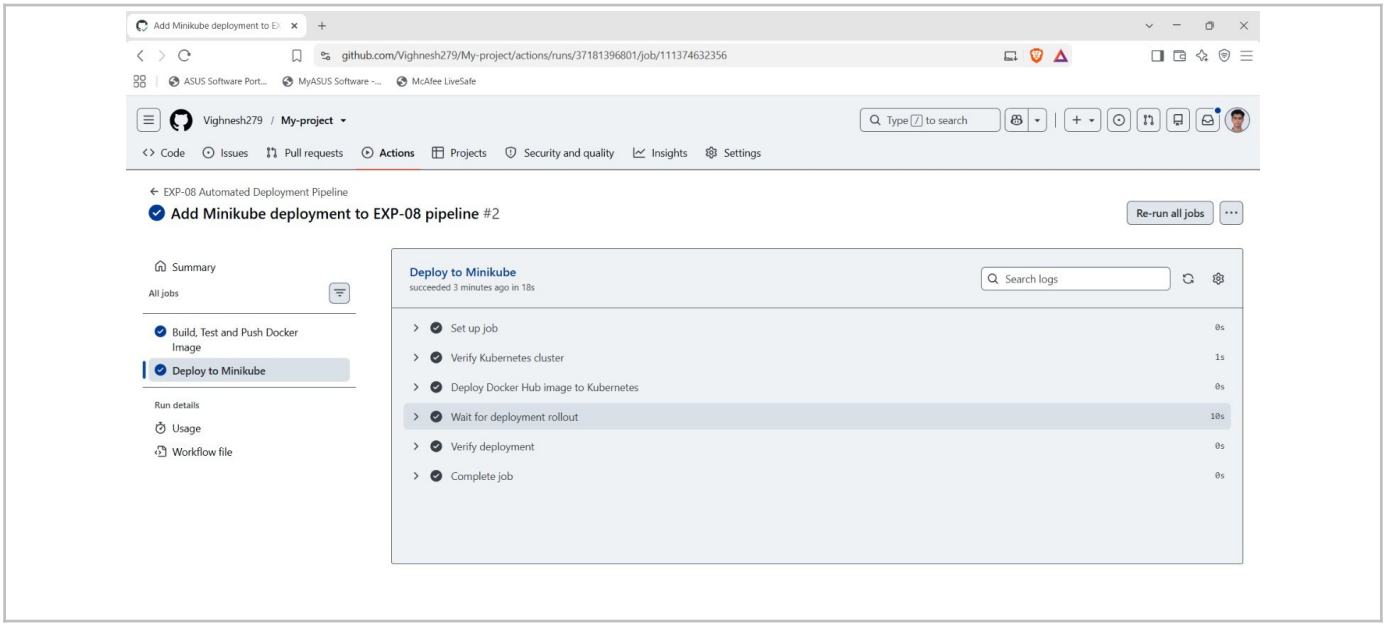
C:\Users\28dip\My-project>git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 749 bytes | 374.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Vighnesh279/My-project.git
   94244ee..c917c9a  main -> main

C:\Users\28dip\My-project>
```

4. Pipeline Execution Output

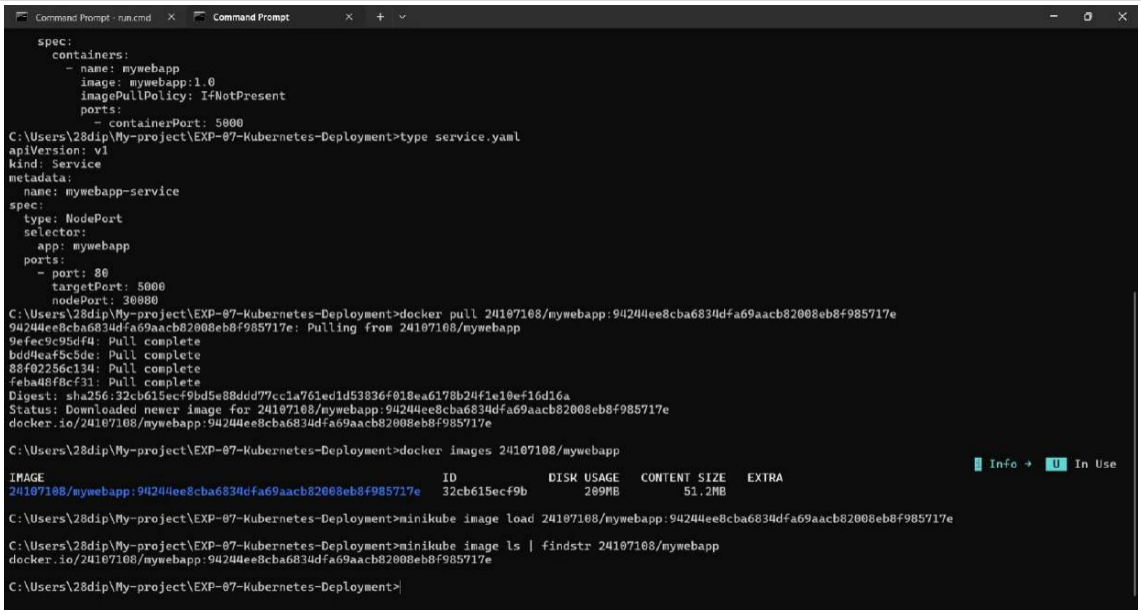
Paste screenshot/output showing successful GitHub Actions/Jenkins pipeline stages for build, test, image push and deployment.





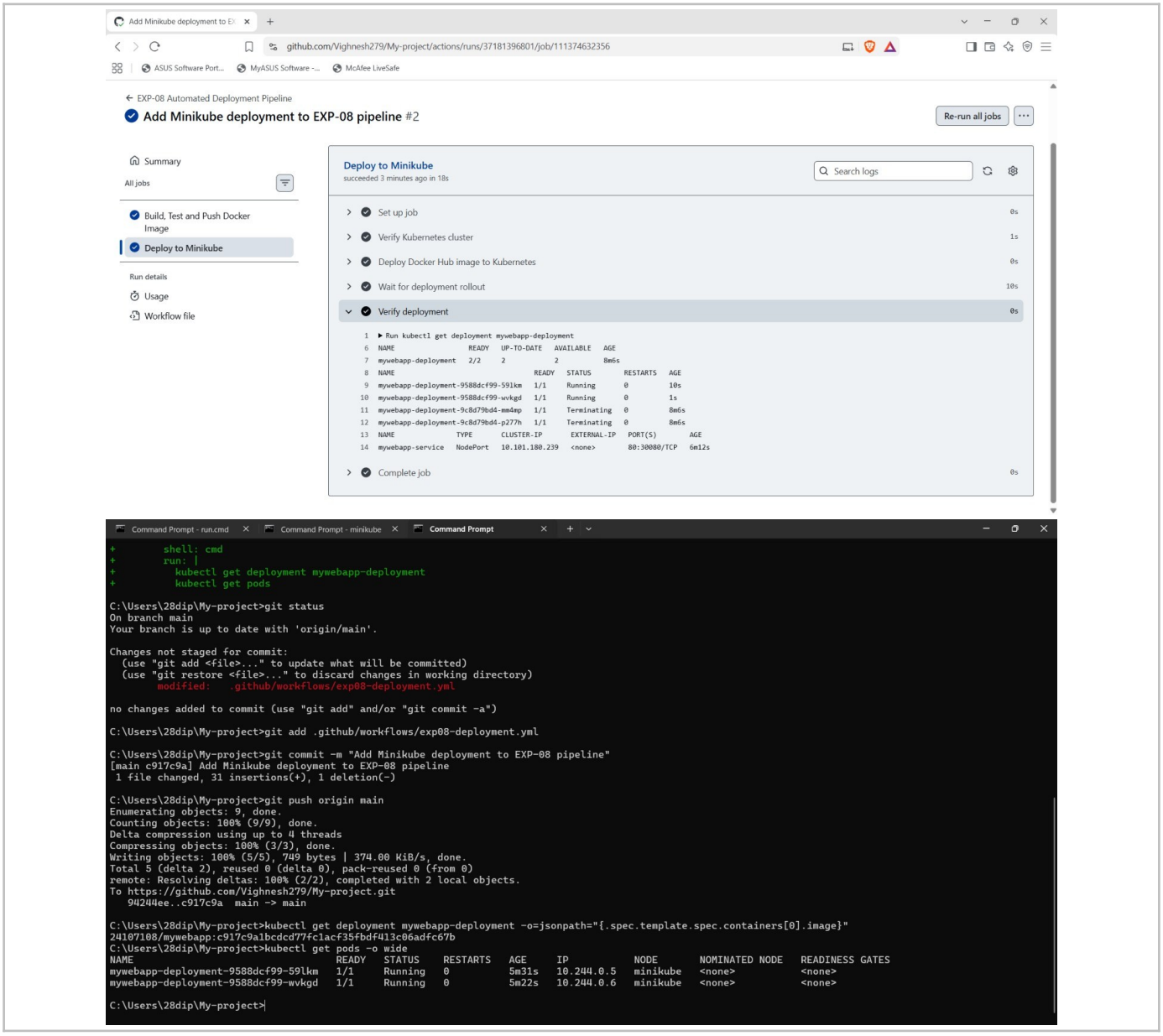
5. Container Registry Output

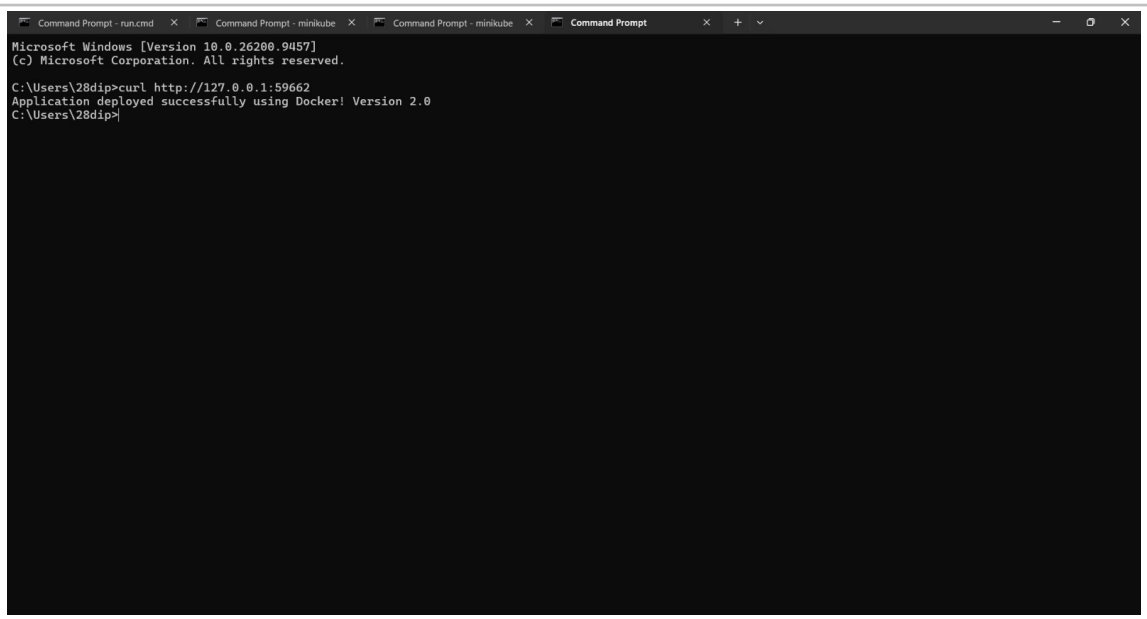
Paste screenshot/output showing the pushed Docker image/tag in Docker Hub or the selected container registry.



6. Deployment Verification Output

Paste screenshot/output showing Kubernetes rollout/server deployment status and application response from browser or curl.





GitHub Repository Link:

https://github.com/Vighnesh279/My-project (folder: EXP-08-Automated-Deployment-Pipeline)

GitHub Actions / Jenkins Pipeline Link:

https://github.com/Vighnesh279/My-project/actions/runs/37181396801 (workflow: EXP-08 Automated Deployment Pipeline)

Docker Hub / Container Registry Image Link:

https://hub.docker.com/r/24107108/mywebapp (image tagged with commit SHA)

Deployment URL / Endpoint:

http://127.0.0.1:59662 (Minikube NodePort service mywebapp-service, port 80:30080)

Final Observation Statement:

```
A GitHub Actions workflow was created that runs the tests, builds the Docker image, pushes it
to Docker Hub as 24107108/mywebapp tagged with the commit SHA, and then deploys it to a local
Minikube cluster using a self-hosted Windows runner. Docker credentials were stored as
repository secrets, and the deployment was verified using the rollout status, kubectl output
and a curl response from the running application.
```

Expected Output

After tests pass on the main branch, the workflow should automatically build the application image, tag it, push it to the container registry and deploy it to the selected target environment. The final application should be accessible through the deployment URL or endpoint.

Conclusion

An automated deployment pipeline was implemented using GitHub Actions or Jenkins. The experiment demonstrated how validated code changes can be packaged, published and deployed automatically, completing a commit-to-deploy DevOps workflow.

Viva / Review Questions

1. How are secrets handled securely in a CD pipeline?
2. What is a rolling update and why is it preferred?
3. Why should Docker images be tagged with a commit SHA instead of only latest?
4. Differentiate Continuous Delivery and Continuous Deployment.

Marks: ____ / 10 **Date: _____** **Signature of Faculty: _____**